openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Secrets Across Clusters

Ahmad Saufi Maulana

Yogyakarta | October, 2026

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Managing Kubernetes secrets with the External Secrets Operator

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

What I wanted from this setup.

Nothing magical. Just a secret flow that a platform team can explain, operate, and rotate without a spreadsheet.

1. **Change it once**
   The credential starts in Vault instead of being copied into every cluster by hand.
2. **Keep Git boring**
   Pull requests contain paths and mappings, not passwords or tokens.
3. **Leave apps alone**
   Existing workloads can keep reading the Kubernetes Secret API they already use.
4. **See the rotation**
   I can tell which cluster updated and whether the application picked up the change.

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

A Kubernetes Secret is still sensitive data inside the cluster.

Base64 is not encryption.

ESO does not change that. It improves how the value arrives and how it is refreshed.

Encrypt API data at rest - Configure encryption for Secret data stored through the Kubernetes API.

Keep RBAC narrow - Only the controller and the intended workloads should be able to read the Secret.

Mount only what is needed - Do not hand an entire secret bundle to a Pod that needs one key.

Audit and practise rotation - A policy is useful only if the team has tested the replacement path.

The manual approach falls apart as clusters multiply.

1 secret × N clusters × M applications

With one cluster, manual work is fine.

With five, every rotation becomes a checklist,

and **eventually** one copy gets missed.

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

We tried the obvious options first.

Manual Secrets

Fine for the first test. Then tickets, copy-and-paste work, and rotation windows begin to pile up.

Vault Agent or sidecar

Very capable, especially for templates and leases, but every workload inherits another runtime component.

Provider SDK in the app

Useful when the app needs Vault features directly. Expensive when every team must learn authentication, renewal, and failure handling.

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Vault keeps the value. ESO handles the Kubernetes hand-off.

Vault stores and governs the credential. ESO brings it into the cluster. The application reads a normal Kubernetes Secret.

1. Vault is still the source

Policy, versions, audit events, and rotation start there.

2. ESO does the sync

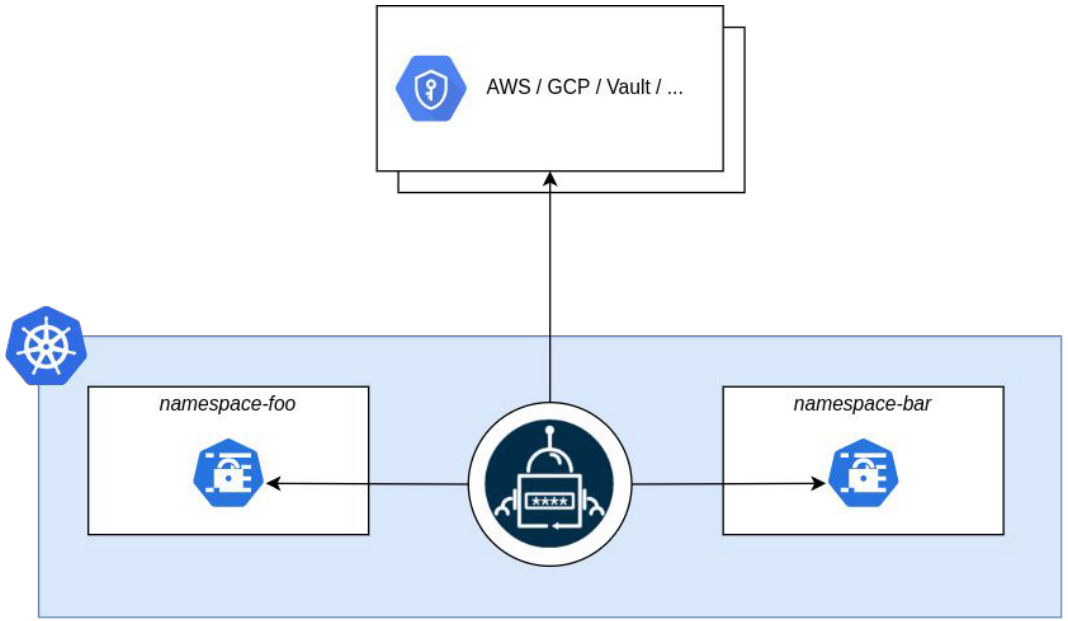
It watches the requested state, reads Vault, and writes the target Secret.

3. The app stays Kubernetes-native

No Vault SDK or sidecar is required for this particular pattern.

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

ESO is the controller in the middle.



openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

ESO is the controller in the middle.

You declare which remote value you want. ESO keeps a Kubernetes Secret in sync with that declaration.

1. Watch

ESO watches resources such as **ExternalSecret**. It runs beside the apps, not inside them.

2. Read

It uses the chosen store and identity to fetch a value from Vault or another provider.

3. Write

It creates or updates the Kubernetes Secret named in the desired state.

The three objects I keep straight.

SecretStore

A namespaced provider connection. It says where the provider is and how ESO authenticates from that namespace.

ClusterSecretStore

The reusable, cluster-scoped form. It is convenient, but its reach deserves tighter review and policy.

ExternalSecret

The actual request: remote path, key mapping, target Secret, and refresh behaviour.

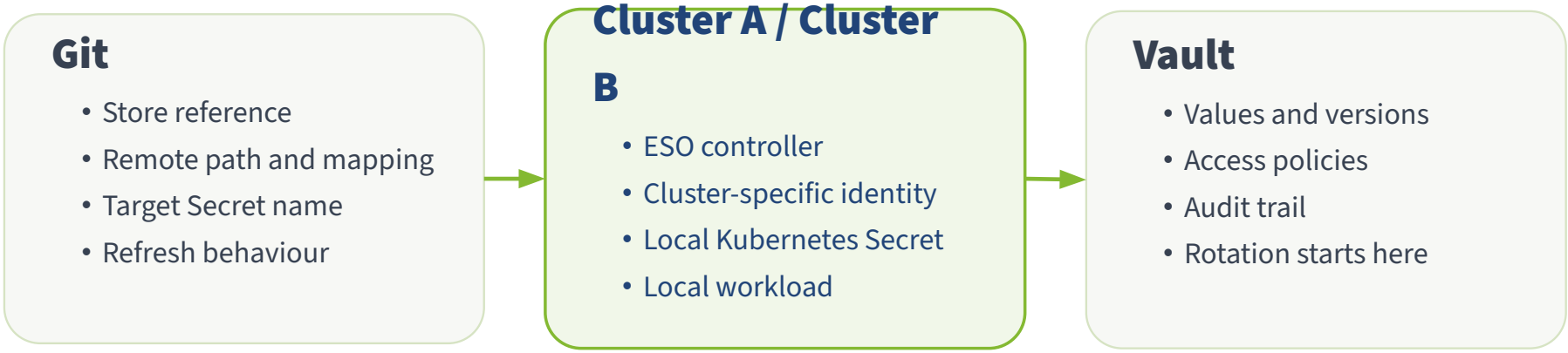
openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

What happens after I apply an ExternalSecret?

1. **Apply**
   GitOps or kubectl sends the desired resource to the cluster.
2. **Notice**
   The controller sees the new or changed resource and queues a reconciliation.
3. **Authenticate**
   ESO resolves the store and uses that cluster or namespace identity with Vault.
4. **Read and map**
   It fetches the remote key and maps the requested fields into Secret data.
5. **Write and report**
   ESO updates the target Secret and records success or failure in status.

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Same Vault, separate cluster identities.



openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

What I commit, and what I do not.

Git gets the recipe

- The store reference
- The remote path and field mapping
- The target Secret name
- The refresh interval and ownership labels

```
spec:
  refreshInterval: 30s
  dataFrom:
  - extract:
      key: platform/demo
```

Vault keeps the ingredient

- The credential value
- Previous and current versions
- The read policy for each cluster
- The provider-side audit record

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

A rotation is finished only when the application has moved.

1. **Vault has v1**
   The current credential is available to both scoped cluster identities.
2. **ESO syncs v1**
   Each cluster has its own Kubernetes Secret containing that version.
3. **Write v2**
   The operator changes the value once, at the source.
4. **Clusters refresh**
   Each ESO instance updates its local Secret on its own schedule.
5. **The app moves**
   The process reloads, restarts, or reopens the credential as designed.

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Six things I would not skip.

- **Encrypt Secret data at rest**
  Protect what the API server persists in etcd.
- **Give each cluster a small Vault policy**
  Only the paths and operations that cluster needs.
- **Trim Kubernetes RBAC**
  ESO writes the intended Secrets; workloads read only their own.
- **Protect the network path**
  Use TLS and decide which namespaces and clusters can reach Vault.
- **Alert on failed syncs**
  An old value can look healthy until the next rotation.
- **Plan rollback**
  Know how to restore a previous value when a consumer rejects the new one.

What should happen when the happy path breaks?

Problem	What you are likely to see	What I would prepare
Vault is down	Reconciliation fails while the previous local Secret may remain.	Alert on status, define how stale is acceptable, and rehearse recovery.
The Vault policy is wrong	ESO gets an authorization error for the requested path.	Keep policies small and make access changes reviewable.
The path or key changed	The target is incomplete or the resource reports an error.	Validate manifests in CI and monitor resource conditions.
The app never reloads	The Secret is new, but the process still uses the old credential.	Choose a reload hook, rollout, or file-watching contract.
The operator is compromised	Its Vault policy and Kubernetes RBAC define the blast radius.	Reduce permissions and verify what you install.

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

DEMO

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

The demo has one job: prove the update path.

1 → 2 → 3 → 4 → 5

Start with two clusters

Cluster A and Cluster B both point to the same demo path.

Check Ready

Confirm the two ExternalSecret resources and their target Secrets.

Change one value

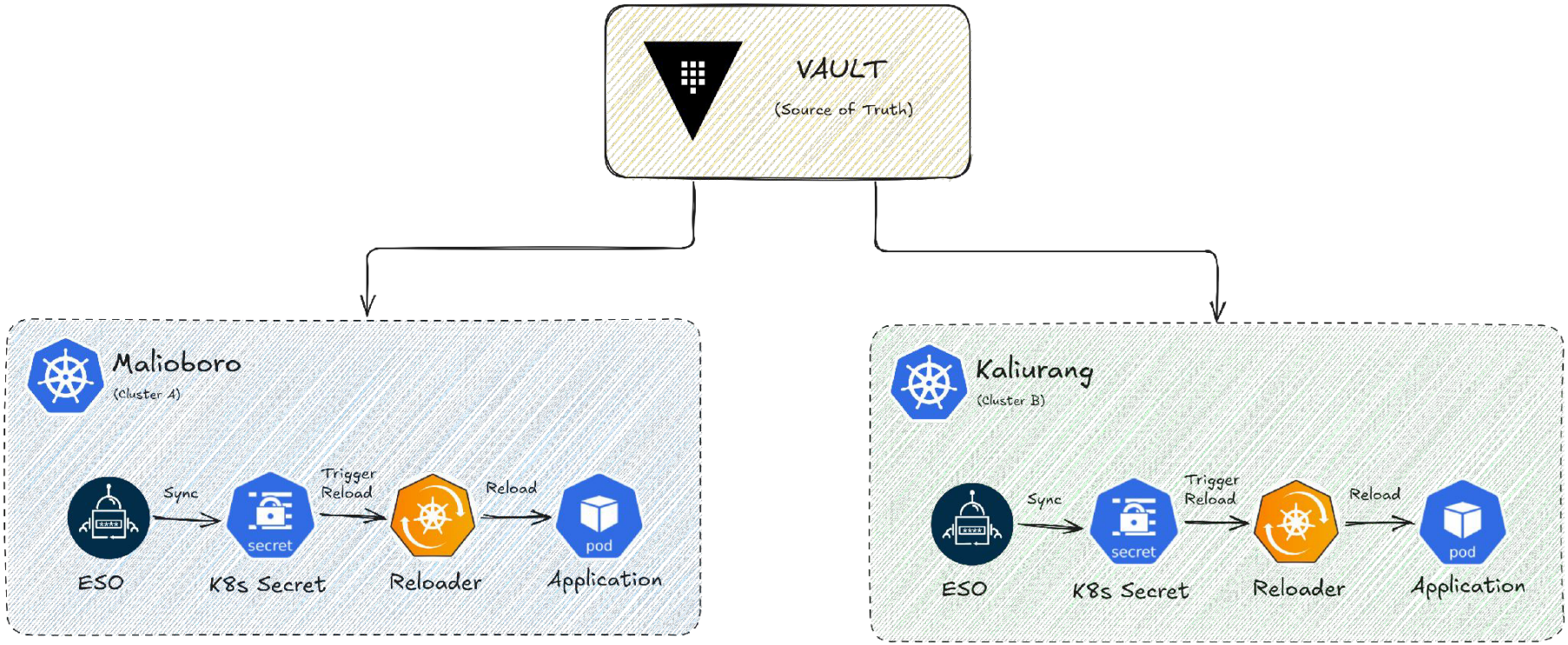
Write a synthetic update in Vault: conference-v1 → conference-v2.

Watch both sync

Follow status and resource versions until both clusters catch up.

Check the consumer

Show what the observer workload sees after the refresh.


VAULT
(Source of Truth)
Malioboro
(Cluster A)
Sync
Trigger Reload
Reload
secret
pod
ESO
K8s Secret
Reloader
Application
Kaliurang
(Cluster B)
Sync
Trigger Reload
Reload
secret
pod
ESO
K8s Secret
Reloader
Application

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

What we saw, and what is still our job.

We saw

- One value stayed under Vault control.
- Git held a reference, not the value.
- Each cluster reconciled independently.
- The rotation was visible from source to consumer.

Still our responsibility

- Encryption at rest and careful RBAC.
- A reliable reload or restart mechanism.
- Vault availability, policy testing, and rollback.
- The operator chart and image supply chain.

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Pick the integration that matches the application.

External Secrets Operator

For applications and Helm charts that already expect a named Kubernetes Secret. It fits GitOps well and keeps provider code out of the app.

Existing Secret contract

Secrets Store CSI driver

For software that naturally reads credentials or certificates from files, especially when you do not want a normal Secret object by default.

Mounted files

Vault Agent / sidecar

For rendered configuration, short-lived credentials, or a process that needs lease renewal close to the workload.

Runtime files and leases

Direct SDK

For a custom service that genuinely needs provider-aware caching, renewal, per-request access, or application-specific fallback logic.

Provider-aware application

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

If the chart supply chain matters, look at SUSE Application Collection.

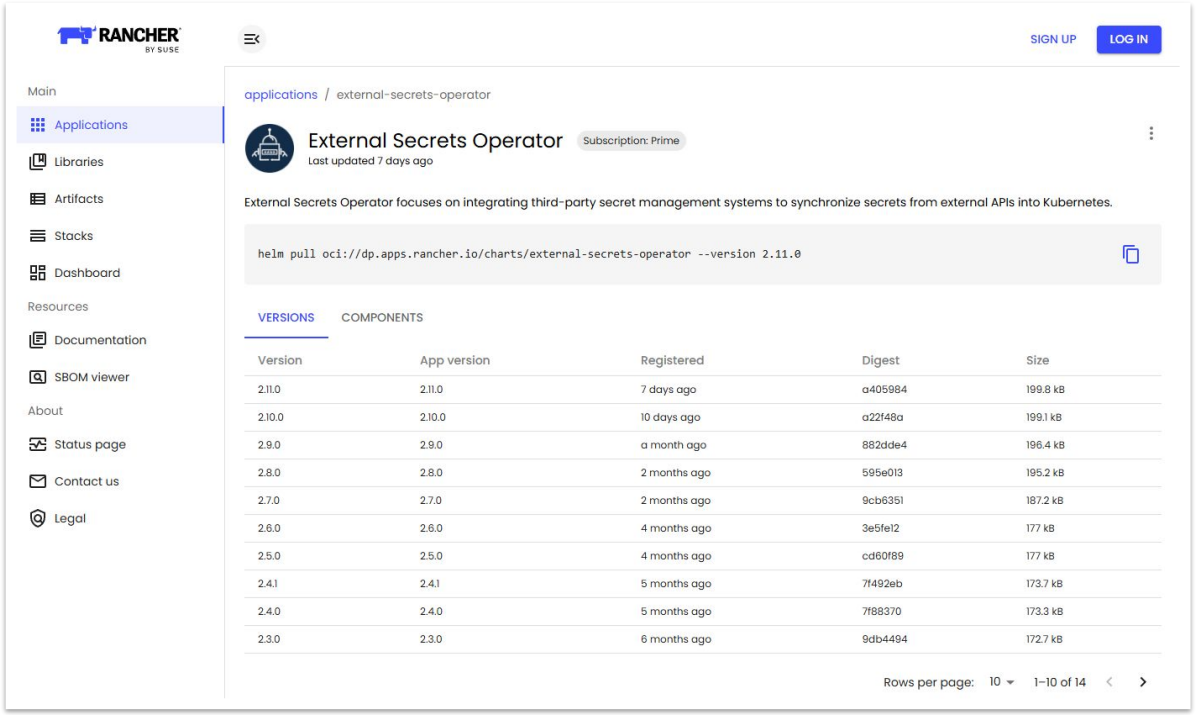
- ESO is delivered as an OCI Helm chart with the container images it deploys.
- The chart, images, and attestations are Sigstore-signed and can be checked with Cosign.
- SBOM and SLSA Level 3 provenance metadata give security teams something concrete to inspect.
- CVE scanning, upstream-driven updates, and SUSE Linux Enterprise base images reduce packaging work for the platform team.

Why I would highlight it

It gives you a curated, verifiable Helm package instead of asking every team to assemble its own chart, images, and evidence. I would still pin the version, verify the signature, review the values, test upgrades, and keep ESO's RBAC and Vault policy small.

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

If the chart supply chain matters, look at SUSE Application Collection.



openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

This is the short version.

01

Keep the value in one source.

Let each cluster read it through its own small, reviewable identity.

02

ESO is a sync controller, not a vault.

It makes external values usable through the Kubernetes Secret interface.

03

Follow rotation into the process.

The work is not done when the Secret object's resource version changes.

04

Choose both the integration and the package deliberately.

Workload behaviour picks the pattern; provenance helps you trust what you install.

openSUSE Asia Summit 2026
Yogyakarta, October 3 - 4

Docs I used and good places to continue.

Kubernetes Secrets
kubernetes.io/docs/concepts/configuration/secret/

Good practices for Kubernetes Secrets
kubernetes.io/docs/concepts/security/secrets-good-practices/

External Secrets Operator overview
external-secrets.io/latest/introduction/overview/

ESO HashiCorp Vault provider
external-secrets.io/latest/provider/hashicorp-vault/

Vault Kubernetes authentication
developer.hashicorp.com/vault/docs/auth/kubernetes

ESO in SUSE Application Collection
apps.rancher.com/applications/external-secrets-operator

Verify Application Collection signatures
docs.apps.rancher.io/howto-guides/verify-signatures-with-cosign

SUSE Application Collection security metadata
suse.com/products/rancher/application-collection/

Thank You



openSUSE®

Asia Summit 2026